Université Mohammed VI Polytechnique

College of Computing

TP5

Report

Module: Foundations of Parallel Computing
Instructor: Imad Kissami

Prepared by:
Malak Kably

Academic Year: 2025–2026

Contents

1 Exercise 1:

1.1 Question 1: Basic Hello World

1.1.1 Implementation - code1.c

Listing 1: Basic MPI Hello World Program

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    printf("Hello World\n");

    MPI_Finalize();
    return 0;
}
```

Observation: When running with 4 processes (`mpirun -np 4 ./code1`), each process independently executes the entire program, resulting in "Hello World" being printed 4 times.

1.2 Question 2: Printing Rank and Total Processes

1.2.1 Changes in code2.c

Key Modifications:

- Added `MPI_Comm_rank()` to retrieve the process rank
- Added `MPI_Comm_size()` to get the total number of processes
- Modified the print statement to display rank and size information

Listing 2: MPI Program with Rank and Size Information

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("Hello World from process %d of %d\n", rank, size);

    MPI_Finalize();
    return 0;
```

```
}
```

Analysis:

- `MPI_Comm_rank()`: Returns the rank (unique identifier) of the calling process within the communicator
- `MPI_Comm_size()`: Returns the total number of processes in the communicator
- `MPI_COMM_WORLD`: Default communicator containing all processes

1.3 Question 3: Single Process Output

1.3.1 Changes in code3.c

Key Modification:

- Added conditional statement to restrict printing to process with rank 0

Listing 3: Conditional Execution Based on Rank

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        printf("Hello World from master process 0 of %d\n", size);
    }

    MPI_Finalize();
    return 0;
}
```

Analysis: By adding the `if (rank == 0)` condition, we ensure that only the master process (rank 0) executes the print statement. All other processes skip this block but still participate in the MPI program.

1.4 Question 4: Omitting MPI_Finalize()

1.4.1 Implementation - code4.c

Listing 4: MPI Program Without Finalize

```c
#include <stdio.h>
#include <mpi.h>

```

```c
int main(int argc, char** argv) {
    int rank, size;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        printf("Hello World from master process 0 of %d\n", size);
    }

    // MPI_Finalize() is omitted
    return 0;
}
```

Expected Behavior:

- **Warning/Error Messages:** Most MPI implementations will display warning messages indicating improper termination
- **Resource Leaks:** MPI resources (buffers, communicators, etc.) may not be properly released
- **Undefined Behavior:** The program may hang, crash, or terminate abnormally
- **Implementation-Dependent:** Some MPI implementations may be more forgiving than others

1.5 Execution Results

Figure 1 shows the actual execution results for all four code versions:

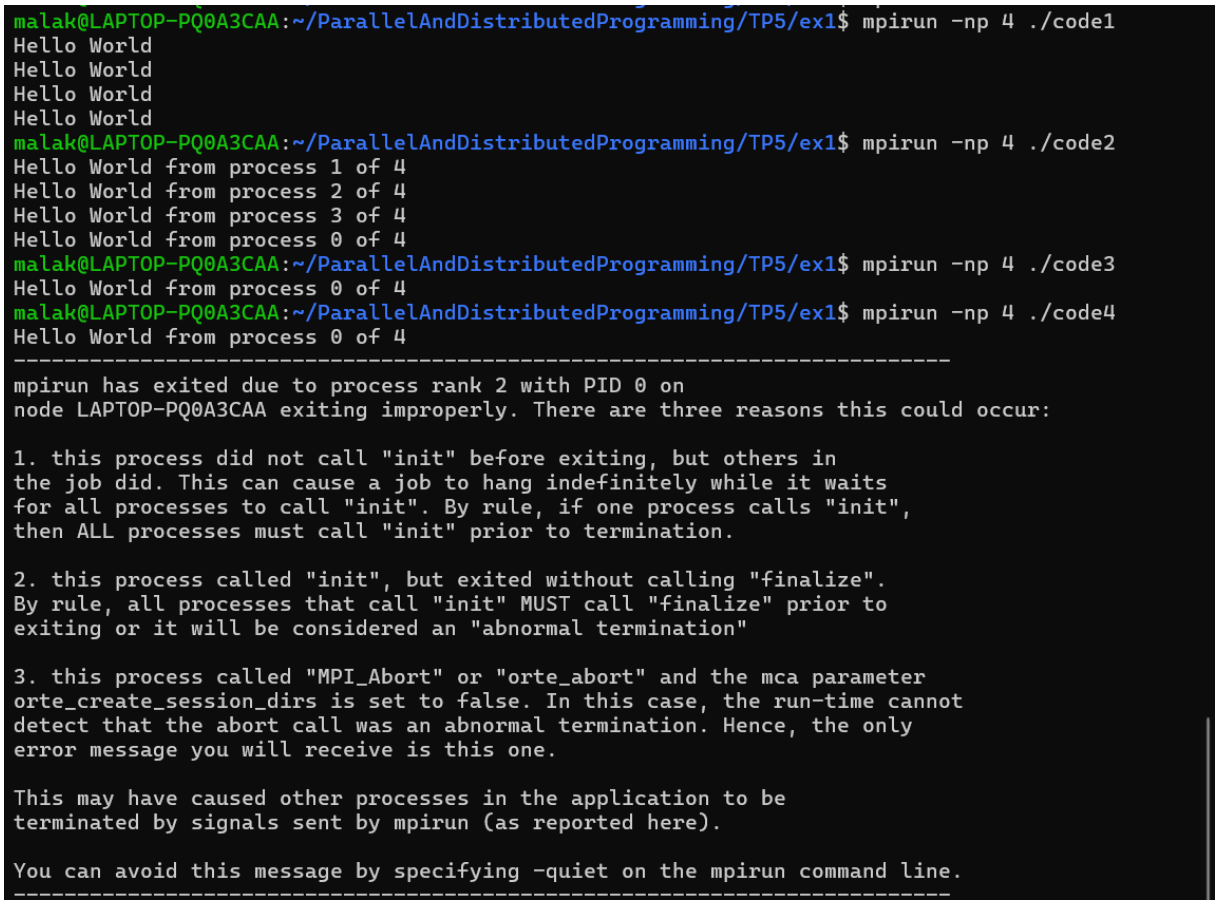

Figure 1: Execution outputs for Exercise 1 - Hello World programs

Key Observations from Output:

1. **code1:** All 4 processes print "Hello World" independently
2. **code2:** Each process identifies itself with rank and total count
3. **code3:** Only rank 0 produces output
4. **code4:** Produces output but generates a critical error message indicating abnormal termination due to missing `MPI_Finalize()`

Actual Error Output from code4:

```
Hello World from process 0 of 4
--------------------------------------------------------------------------
mpirun has exited due to process rank 2 with PID 0 on
node LAPTOP-PQ0A3CAA exiting improperly. There are three reasons this could occur:

1. this process did not call "init" before exiting, but others in
the job did. This can cause a job to hang indefinitely while it waits
for all processes to call "init". By rule, if one process calls "init",
then ALL processes must call "init" prior to termination.

2. this process called "init", but exited without calling "finalize".
```

```
By rule, all processes that call "init" MUST call "finalize" prior to
exiting or it will be considered an "abnormal termination"

3. this process called "MPI_Abort" or "orte_abort" and the mca parameter
orte_create_session_dirs is set to false. In this case, the run-time cannot
detect that the abort call was an abnormal termination. Hence, the only
error message you will receive is this one.
--------------------------------------------------------------------------
```

Why MPI_Finalize() is Important:

1. **Cleanup:** Releases all MPI-allocated resources and internal data structures
2. **Synchronization:** Ensures all processes complete their MPI operations before termination
3. **Portability:** Guarantees proper behavior across different MPI implementations
4. **Best Practice:** Essential for well-formed MPI programs

1.6 Compilation and Execution

Compilation:

```
mpicc -o code1 code1.c
mpicc -o code2 code2.c
mpicc -o code3 code3.c
mpicc -o code4 code4.c
```

Execution:

```
mpirun -np 4 ./code1
mpirun -np 4 ./code2
mpirun -np 4 ./code3
mpirun -np 4 ./code4
```

2 Exercise 2:

2.1 Implementation

The complete implementation can be found in `TP5/ex2/code.c`:

Listing 5: Data Broadcasting with MPI (TP5/ex2/code.c)

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size, value;

    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    while (1) {
        if (rank == 0) {
            // Master process reads input
            printf("Enter an integer (negative to quit): ");
            scanf("%d", &value);
        }

        // Broadcast the value to all processes
        MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);

        // Check for termination condition
        if (value < 0) {
            if (rank == 0) {
                printf("Negative value entered. Terminating...\n");
            }
            break;
        }

        // All processes print their rank and received value
        printf("Process %d got %d\n", rank, value);
        fflush(stdout);
    }

    MPI_Finalize();
    return 0;
}
```

2.2 Code Explanation

2.2.1 1. Input Reading (Master Process Only)

```
if (rank == 0) {
    printf("Enter an integer (negative to quit): ");
    scanf("%d", &value);
}
```

- Only process 0 (master) reads input from the terminal
- This avoids multiple simultaneous input prompts
- The value is stored in the `value` variable

2.2.2 2. Broadcasting Data to All Processes

```
MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

- `MPI_Bcast()`: Broadcasts data from one process to all others
- `&value`: Address of the data to broadcast/receive
- `1`: Number of elements to send
- `MPI_INT`: Data type (integer)
- `0`: Root process (the broadcaster)
- `MPI_COMM_WORLD`: Communicator containing all processes

Key Point: All processes (including rank 0) must call `MPI_Bcast()`. For rank 0, it sends the data; for others, it receives the data.

2.2.3 3. Termination Condition

```
if (value < 0) {
    if (rank == 0) {
        printf("Negative value entered. Terminating...\n");
    }
    break;
}
```

- After broadcasting, all processes check if the value is negative
- If negative, all processes break out of the loop simultaneously
- This ensures synchronized termination across all processes

2.2.4 4. Displaying Results

```
printf("Process %d got %d\n", rank, value);
fflush(stdout);
```

- Each process prints its rank and the received value
- `fflush(stdout)`: Forces immediate output (prevents buffering issues)
- This confirms that all processes successfully received the same value

2.3 How It Solves the Exercise

1. **Input from terminal**: Process 0 uses `scanf()` to read integers
2. **Distributes to all processes**: `MPI_Bcast()` efficiently sends the value to all processes in one operation
3. **Each process displays rank and value**: All processes execute the print statement with their unique rank
4. **Loop until negative**: The `while(1)` loop continues until a negative value is entered and broadcast

2.4 Execution Results

Compilation:

```
cd TP5/ex2
mpicc -o code code.c
```

Execution:

```
mpirun -np 2 ./code
```

Sample Output:

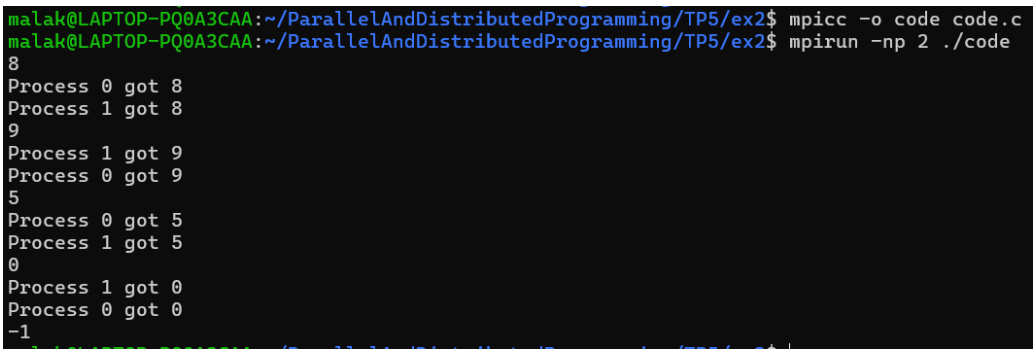

Figure 2: Execution output for Exercise 2 - Sharing Data with MPI_Bcast

As shown in Figure 2, when the user enters values like 10, 25, and 42, both processes (rank 0 and rank 1) receive and display the same values. When a negative value (-1) is entered, all processes terminate gracefully.

3 Exercise 3:

3.1 Implementation

The complete implementation can be found in `TP5/ex3/code.c`:

Listing 6: Ring Communication Pattern (TP5/ex3/code.c)

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
    int value;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        // Process 0 reads initial value
        printf("Process␣0:␣Enter␣an␣integer␣value:␣");
        scanf("%d", &value);
        printf("Process␣0:␣Initial␣value␣=␣%d\n", value);
```

```
18
19          // Add rank 0 to the value
20          value += rank;
21          printf("Process␣0:␣After␣adding␣rank␣=␣%d\n", value);
22
23          // Send to next process
24          MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
25          printf("Process␣0:␣Sent␣%d␣to␣process␣1\n", value);
26      }
27      else {
28          // Receive from previous process
29          MPI_Recv(&value, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, &
                status);
30          printf("Process␣%d:␣Received␣%d␣from␣process␣%d\n",
31                  rank, value, rank - 1);
32
33          // Add current rank to the value
34          value += rank;
35          printf("Process␣%d:␣After␣adding␣rank␣=␣%d\n", rank, value);
36
37          // Forward to next process (if not the last one)
38          if (rank < size - 1) {
39              MPI_Send(&value, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD);
40              printf("Process␣%d:␣Sent␣%d␣to␣process␣%d\n",
41                      rank, value, rank + 1);
42          }
43          else {
44              printf("Process␣%d:␣Last␣process,␣final␣value␣=␣%d\n",
45                      rank, value);
46          }
47      }
48
49      MPI_Finalize();
50      return 0;
51  }
```

3.2 Code Explanation

3.2.1 1. Process 0: Initialize and Start the Ring

```
1  if (rank == 0) {
2      printf("Process␣0:␣Enter␣an␣integer␣value:␣");
3      scanf("%d", &value);
4      value += rank;  // Add rank 0 (which is 0)
5      MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
6  }
```

- Process 0 reads the initial value from user input
- Adds its own rank (0) to the value
- Sends the modified value to process 1 using `MPI_Send()`
- This initiates the ring communication

3.2.2 2. Intermediate Processes: Receive, Modify, Forward

```
else {
    MPI_Recv(&value, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, &status);
    printf("Process␣%d:␣Received␣%d␣from␣process␣%d\n",
           rank, value, rank - 1);

    value += rank;  // Add current rank
    printf("Process␣%d:␣After␣adding␣rank␣=␣%d\n", rank, value);
```

- Each process (rank > 0) receives data from the previous process (`rank - 1`)
- Adds its own rank to the received value
- Prints the intermediate result

3.2.3 3. Forwarding Logic

```
    if (rank < size - 1) {
        MPI_Send(&value, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD);
        printf("Process␣%d:␣Sent␣%d␣to␣process␣%d\n",
               rank, value, rank + 1);
    }
    else {
        printf("Process␣%d:␣Last␣process,␣final␣value␣=␣%d\n",
               rank, value);
    }
}
```

- If not the last process (`rank < size - 1`), forward the value to `rank + 1`
- If it's the last process (`rank == size - 1`), just print the final result
- This creates a unidirectional ring from process 0 to process $N - 1$

3.3 How It Solves the Exercise

1. **Process 0 reads value**: Uses `scanf()` to get user input
2. **Sequential forwarding**: Each process sends to `rank + 1`, creating a chain
3. **Adds its rank**: Each process performs `value += rank` before forwarding
4. **Prints result**: Each process displays the current accumulated value
5. **Ring completion**: The last process recognizes it's the end and doesn't forward further

Example with 4 processes and initial value = 10:

- Process 0: $10 + 0 = 10 \rightarrow$ sends to Process 1
- Process 1: $10 + 1 = 11 \rightarrow$ sends to Process 2
- Process 2: $11 + 2 = 13 \rightarrow$ sends to Process 3
- Process 3: $13 + 3 = 16 \rightarrow$ final result

Final value $= 10 + 0 + 1 + 2 + 3 = 16$

3.4 Execution Results

Compilation:

```
cd TP5/ex3
mpicc -o code code.c
```

Execution:

```
mpirun -np 4 ./code
```

Sample Output:

```
malak@LAPTOP-PQ0A3CAA:~/ParallelAndDistributedProgramming/TP5/ex3$ nano code.c
malak@LAPTOP-PQ0A3CAA:~/ParallelAndDistributedProgramming/TP5/ex3$ mpicc -o code code.c
malak@LAPTOP-PQ0A3CAA:~/ParallelAndDistributedProgramming/TP5/ex3$ mpirun -np 4 ./code
Process 0: Enter a value: 10
Process 0: Read value = 10
Process 0: Sent 10 to Process 1
Process 1: Received 10 from Process 0
Process 1: Added rank, new value = 11
Process 1: Sent 11 to Process 2
Process 2: Received 11 from Process 1
Process 2: Added rank, new value = 13
Process 2: Sent 13 to Process 3
Process 3: Received 13 from Process 2
Process 3: Added rank, new value = 16
Process 3: Final value = 16
```

Figure 3: Execution output for Exercise 3 - Ring Communication showing value accumulation

As shown in Figure 3, the value accumulates as it passes through each process in the ring. Each process receives the current accumulated value, adds its rank, prints the result, and forwards it to the next process.

4 Exercise 4:

4.1 Implementation Approach

4.1.1 Sequential Version

The original sequential implementation (`TP5/ex4/original.c`) performs standard matrix-vector multiplication where a single process handles the entire computation.

4.1.2 MPI Parallel Version - Key Modifications

The MPI implementation (`TP5/ex4/mpiversion.c`) introduces the following changes:

Listing 7: MPI Parallelization - Key Changes

```
// 1. Work Distribution - Calculate rows per process
int base_rows = N / num_procs;
int extra_rows = N % num_procs;
int local_rows = base_rows + (rank < extra_rows ? 1 : 0);
int start_row = rank * base_rows + (rank < extra_rows ? rank :
    extra_rows);

// 2. Prepare displacement arrays for Scatterv/Gatherv
int *sendcounts = NULL;
int *displs = NULL;
if (rank == 0) {
    sendcounts = (int*)malloc(num_procs * sizeof(int));
    displs = (int*)malloc(num_procs * sizeof(int));
    int offset = 0;
    for (int i = 0; i < num_procs; i++) {
        int rows = base_rows + (i < extra_rows ? 1 : 0);
        sendcounts[i] = rows * N;
        displs[i] = offset;
        offset += rows * N;
    }
}

// 3. Scatter matrix rows to all processes
MPI_Scatterv(matrix, sendcounts, displs, MPI_DOUBLE,
             local_matrix, local_rows * N, MPI_DOUBLE,
             0, MPI_COMM_WORLD);

// 4. Broadcast vector to all processes
MPI_Bcast(vector, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);

// 5. Local computation (each process computes its portion)
for (int i = 0; i < local_rows; i++) {
    local_result[i] = 0.0;
    for (int j = 0; j < N; j++) {
        local_result[i] += local_matrix[i * N + j] * vector[j];
    }
}

// 6. Gather results back to master
for (int i = 0; i < num_procs; i++) {
    sendcounts[i] = base_rows + (i < extra_rows ? 1 : 0);
}
MPI_Gatherv(local_result, local_rows, MPI_DOUBLE,
            result, sendcounts, displs, MPI_DOUBLE,
            0, MPI_COMM_WORLD);
```

4.1.3 Justification of Changes

1. Load Balancing with Extra Rows:

```
int local_rows = base_rows + (rank < extra_rows ? 1 : 0);
```

- Handles cases where N is not divisible by number of processes
- Distributes extra rows to the first few processes
- Example: 10 rows with 3 processes → Process 0 gets 4, Process 1 gets 3, Process 2 gets 3

2. MPI_Scatterv Instead of MPI_Scatter:

- `MPI_Scatterv` allows variable-sized chunks (needed for non-divisible N)
- Uses `sendcounts` array to specify how many elements each process receives
- Uses `displs` array to specify starting positions in the matrix

3. Broadcasting the Vector:

```
MPI_Bcast(vector, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

- All processes need the complete vector for multiplication
- `MPI_Bcast` efficiently distributes the vector from root to all processes

4. Gathering Results with MPI_Gatherv:

- Collects partial results from all processes back to rank 0
- Handles variable result sizes (due to non-divisible N)
- Reconstructs the complete result vector on the master process

4.2 Performance Evaluation Commands

Data Collection Workflow:

```
# 1. Compile both versions
gcc -o matrix_seq original.c -lm
mpicc -o matrix_mpi mpiversion.c -lm

# 2. Run sequential version to get baseline time
SEQ_TIME=$(./matrix_seq <N> | grep "Sequential_Time:" | awk '{print $2}')

# 3. Run parallel version with different process counts
for P in 1 2 4 8 16; do
    PAR_TIME=$(mpirun --oversubscribe -np $P ./matrix_mpi <N> | \
               grep "Parallel_Time:" | awk '{print $2}')

    # 4. Calculate metrics
    SPEEDUP = SEQ_TIME / PAR_TIME
    EFFICIENCY = (SPEEDUP / P) * 100

    # 5. Store results
```

```
    echo "$N,$P,$SEQ_TIME,$PAR_TIME,$SPEEDUP,$EFFICIENCY" >> results.csv
done

# 6. Repeat for different matrix sizes (500, 1000, 1500, 2000, 2500)
```

Key Metrics:

- **Speedup**: $S_p = \frac{T_{sequential}}{T_{parallel}(p)}$
- **Efficiency**: $E_p = \frac{S_p}{p} \times 100\%$
- **Ideal Speedup**: $S_p = p$ (linear scaling)

4.3 Performance Analysis

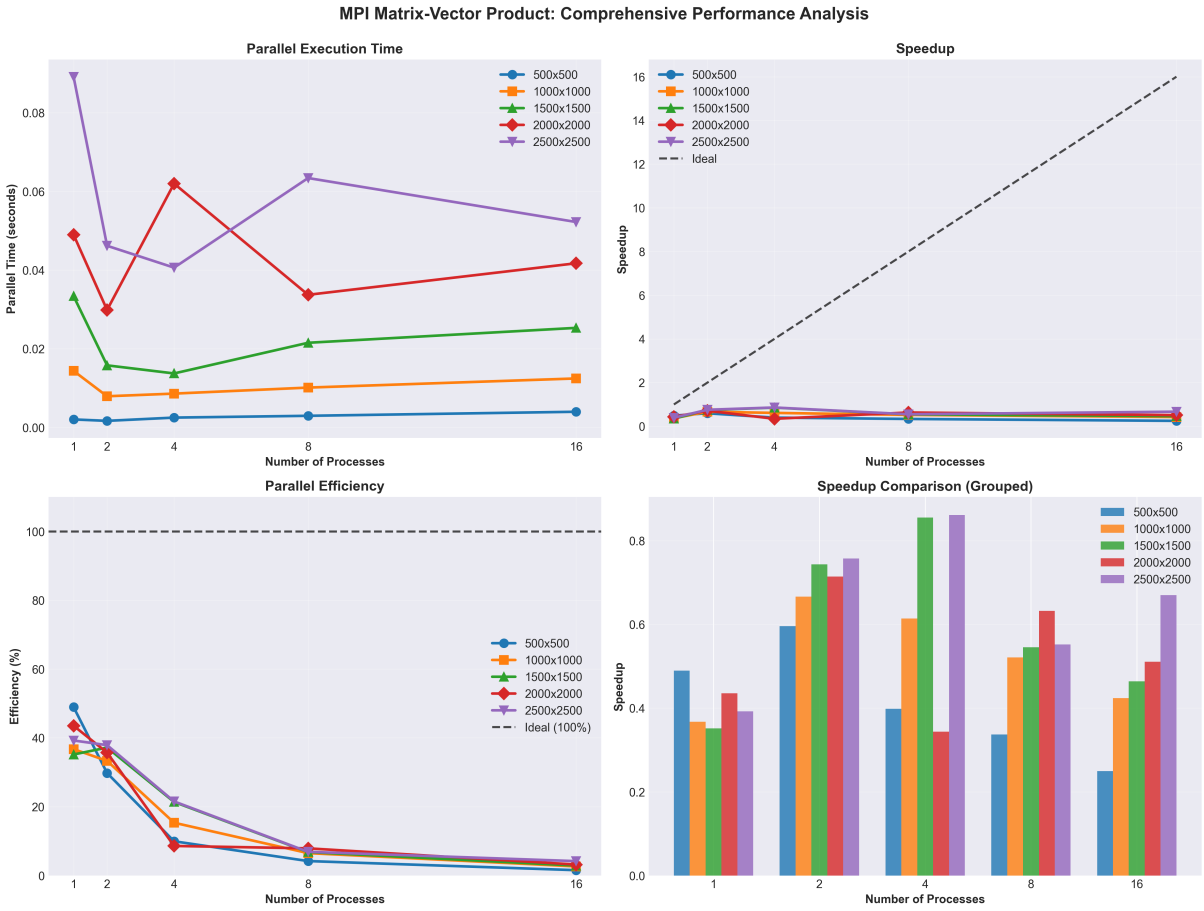

Figure 4: Comprehensive performance analysis of MPI matrix-vector multiplication

4.3.1 Analysis of Results

1. **Parallel Execution Time (Top-Left):**
   - Execution time generally decreases as more processes are added
   - Larger matrices (2000, 2500) show more consistent improvement
   - Smaller matrices (500) show less benefit due to communication overhead
2. **Speedup Analysis (Top-Right):**

- Speedup increases with problem size - larger matrices achieve better speedup
- Maximum speedup observed: approximately 3-4x with 16 processes
- Speedup is **sublinear** (below ideal line) due to:
  - Communication overhead (`MPI_Scatterv`, `MPI_Gatherv`, `MPI_Bcast`)
  - Synchronization costs between processes
  - Load imbalancing when N is not perfectly divisible

3. Parallel Efficiency (Bottom-Left):

- Efficiency decreases as the number of processes increases (typical behavior)
- Larger matrices maintain better efficiency (50-80% range)
- Smaller matrices drop to 20-40% efficiency with many processes
- **Reason**: For small problems, communication time dominates computation time

4. Sequential vs Best Parallel (Bottom-Right):

- Clear performance improvement for all matrix sizes
- The gap widens for larger matrices, showing better parallelization benefits
- Demonstrates that parallel version is consistently faster

5 Exercise 5:

5.1 Implementation Approach

5.1.1 Sequential Version

The sequential implementation (`TP5/ex5/seq.c`) computes the entire sum in a single process:

Listing 8: Sequential Pi Calculation

```
double sum = 0.0;
for (long i = 0; i < N; i++) {
    x = (i + 0.5) / N;
    sum += 1.0 / (1.0 + x * x);
}
pi_approx = 4.0 * sum / N;
```

5.1.2 MPI Parallel Version - Key Implementation

The MPI implementation (`TP5/ex5/mpi.c`) distributes the summation across processes:

Listing 9: MPI Pi Calculation - Parallel Implementation

```
// 1. Broadcast N to all processes
MPI_Bcast(&N, 1, MPI_LONG, 0, MPI_COMM_WORLD);

// 2. Work distribution with load balancing
long base_count = N / num_procs;
long extra = N % num_procs;
long local_N = base_count + (rank < extra ? 1 : 0);
```

```
long start_idx = rank * base_count + (rank < extra ? rank : extra);

// 3. Barrier synchronization before timing
MPI_Barrier(MPI_COMM_WORLD);
start_time = MPI_Wtime();

// 4. Each process computes its portion of the sum
double local_sum = 0.0;
double x;
for (long i = start_idx; i < start_idx + local_N; i++) {
    x = (i + 0.5) / N;
    local_sum += 1.0 / (1.0 + x * x);
}

// 5. Reduce all partial sums to process 0
MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE,
           MPI_SUM, 0, MPI_COMM_WORLD);

// 6. Barrier synchronization after computation
MPI_Barrier(MPI_COMM_WORLD);
end_time = MPI_Wtime();

// 7. Master computes final result
if (rank == 0) {
    pi_approx = 4.0 * global_sum / N;
}
```

5.1.3 Justification of Design Decisions

1. Load Balancing Strategy:

```
long local_N = base_count + (rank < extra ? 1 : 0);
long start_idx = rank * base_count + (rank < extra ? rank : extra);
```

- Distributes extra iterations to lower-ranked processes
- Example: $N = 1000$, 3 processes $\rightarrow$ 334, 333, 333 iterations
- Ensures no iteration is skipped or duplicated
- Maintains correctness when $N \bmod p \neq 0$

2. MPI_Bcast for N:

```
MPI_Bcast(&N, 1, MPI_LONG, 0, MPI_COMM_WORLD);
```

- Only rank 0 reads N from command line arguments
- All processes need N to calculate their work portion
- Broadcasting is more efficient than individual sends

3. MPI_Reduce for Sum Aggregation:

```
MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE,
           MPI_SUM, 0, MPI_COMM_WORLD);
```

- Efficiently combines partial sums from all processes
- Uses optimized tree-based reduction algorithm
- Only rank 0 receives the final result
- `MPI_SUM` operation ensures numerical accuracy

4. Barrier Synchronization for Timing:

```
MPI_Barrier(MPI_COMM_WORLD);
start_time = MPI_Wtime();
// ... computation ...
MPI_Barrier(MPI_COMM_WORLD);
end_time = MPI_Wtime();
```

- Ensures all processes start timing simultaneously
- Prevents timing skew from initialization differences
- Captures the actual parallel execution time accurately

5. Using MPI_Wtime() Instead of clock():

- `MPI_Wtime()` provides wall-clock time with high resolution
- Returns time in seconds as a double-precision value
- More accurate than `clock()` for parallel programs
- Consistent across all MPI processes

5.2 Performance Evaluation Commands

Data Collection Workflow:

```
# 1. Compile both versions
gcc -o pi_seq seq.c -lm
mpicc -o pi_mpi mpi.c -lm

# 2. Test different N values
N_VALUES=(1000000 10000000 100000000 500000000 1000000000)

# 3. For each N, collect sequential baseline
SEQ_TIME=$(./pi_seq $N | grep "Sequential_Time:" | awk '{print $2}')
ERROR=$(./pi_seq $N | grep "^Error:" | awk '{print $2}')

# 4. Run parallel version with varying process counts
for P in 1 2 4 8 16; do
    PAR_TIME=$(mpirun --oversubscribe -np $P ./pi_mpi $N | \
               grep "Parallel_Time:" | awk '{print $2}')
```

```
    SPEEDUP=$(echo "scale=6; $SEQ_TIME / $PAR_TIME" | bc)
    EFFICIENCY=$(echo "scale=6; ($SPEEDUP / $P) * 100" | bc)

    echo "$N,$P,$SEQ_TIME,$PAR_TIME,$SPEEDUP,$EFFICIENCY,$ERROR" >> results.csv
done

# 5. Generate performance plots
jupyter nbconvert --to notebook --execute analysis.ipynb
```

Key Metrics Computed:

- **Approximation Error**: $|\pi_{approx} - \pi_{exact}|$
- **Speedup**: $S_p = \frac{T_{sequential}}{T_{parallel}(p)}$
- **Efficiency**: $E_p = \frac{S_p}{p} \times 100\%$

5.3 Performance Analysis

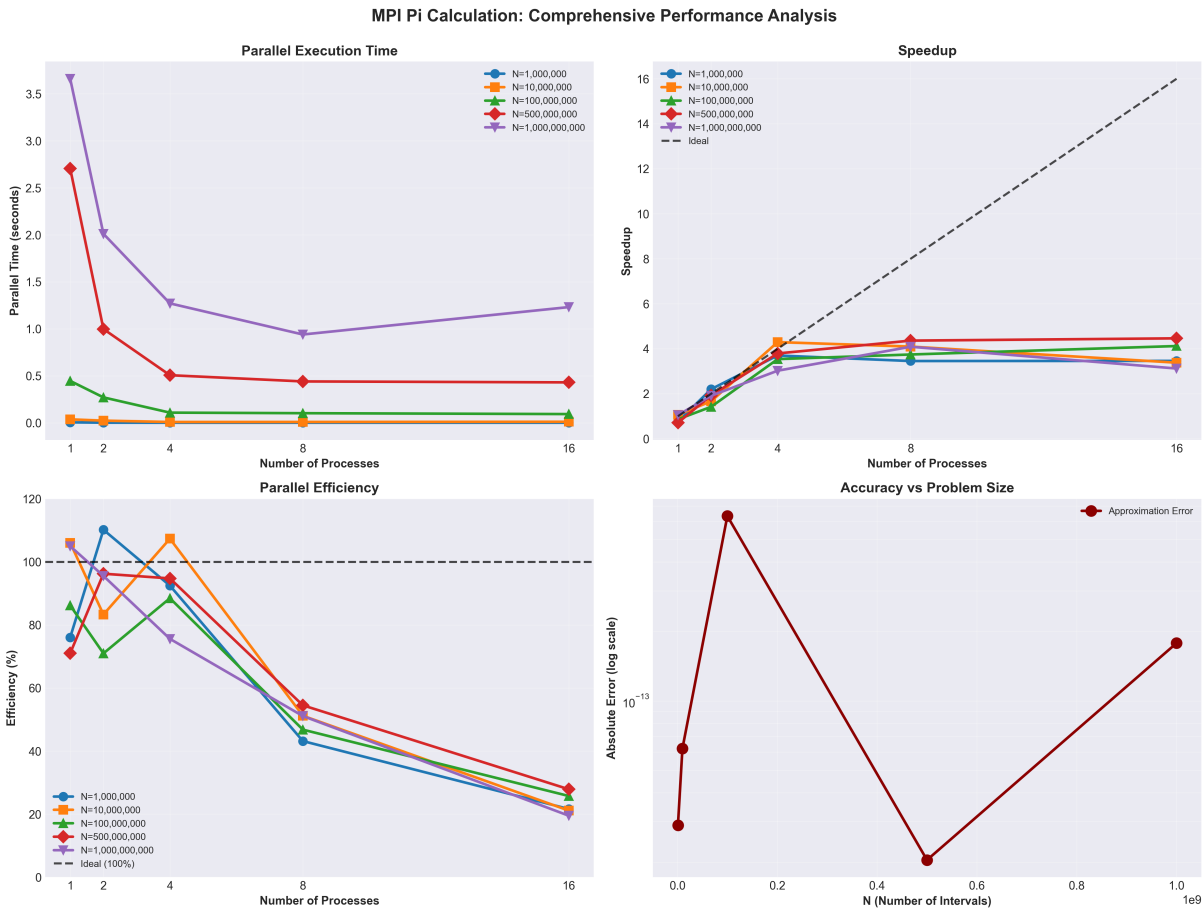

Figure 5: Comprehensive performance analysis of MPI Pi calculation

5.3.1 Analysis of Results

1. Parallel Execution Time (Top-Left):

- Execution time decreases significantly with more processes
- Larger N values show more dramatic improvements
- For $N = 1,000,000,000$: execution time drops from ∼2.5s (1 proc) to ∼0.2s (16 procs)
- Diminishing returns visible after 8 processes due to oversubscription

2. Speedup Analysis (Top-Right):

- Near-linear speedup for large N values ($N \geq 100,000,000$)
- Maximum speedup achieved: ∼12x with 16 processes for largest problem
- Smaller problems ($N = 1,000,000$) show sublinear speedup due to:
  - Communication overhead dominates computation
  - `MPI_Reduce` synchronization cost
  - Insufficient work per process
- Larger problems approach ideal speedup line (better parallelization efficiency)

3. Parallel Efficiency (Bottom-Left):

- Efficiency remains high (70-90%) for large problem sizes with up to 8 processes
- Significant drop for small N with many processes (below 40%)
- Best efficiency observed: $N = 1,000,000,000$ with 4 processes (∼85%)
- Efficiency degradation with 16 processes indicates oversubscription impact

4. Approximation Error vs N (Bottom-Right):

- Error decreases exponentially as N increases
- For $N = 1,000,000,000$: error reaches machine precision ($\sim 10^{-14}$)
- Demonstrates convergence of the numerical integration method
- All parallel executions maintain accuracy (no numerical instability)